

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки (специальность) **09.03.03 Прикладная информатика**

ПРАКТИЧЕСКАЯ РАБОТА № 3

По дисциплине «Алгоритмы и структуры данных»

Тема: Графы.
Вариант 1.

Выполнила	Борзова Д. М.; К3140.
Проверила	Ромакина О. М.
Дата	_____

Санкт-Петербург 2026

Оглавление

Оглавление	2
Задачи по варианту	4
1 Задача. Лабиринт [5 s, 512 Mb]	4
Описание задания к задаче	4
Исходный код и описание решения	4
Описание проведённых тестов	5
Вывод по задаче	7
5 Задача. Город с односторонним движением [5 s, 512 Mb]	8
Описание задания к задаче	8
Исходный код и описание решения	8
Описание проведённых тестов	10
13 Задача. Грядки [1 s, 16 Mb]	12
Описание задания к задаче	12
Исходный код и описание решения	12
Описание проведённых тестов	13
Дополнительные задачи	18
2 Задача. Компоненты [5 s, 512 Mb, 1 балл, с учётом штрафа 0,5 балла]	18
Описание задания к задаче	18
Исходный код и описание решения	18
Описание проведённых тестов	19
3 Задача. Циклы [5 s, 512 Mb, 1 балл, с учётом штрафа 0,5 балла]	21
Описание задания к задаче	21
Исходный код и описание решения	21
Описание проведённых тестов	23
4 Задача. Порядок курсов [10 s, 512 Mb, 1 балл, с учётом штрафа 0,5 балла]	25
Описание задания к задаче	25
Исходный код и описание решения	25
Описание проведённых тестов	26
8 Задача. Стоимость полета [10 s, 512 Mb, 1,5 балла, с учётом штрафа 1 балл]	29
Описание задания к задаче	29
Исходный код и описание решения	29
Описание проведённых тестов	31
9 Задача. Аномалии курсов валют [10 s, 512 Mb, 2 балла, с учётом штрафа 1,5 балла]	34
Описание задания к задаче	34

Исходный код и описание решения	34
Описание проведённых тестов	36
10 Задача. Оптимальный обмен валюты [10 s, 512 Mb, 2 балла, с учётом штрафа 1,5 балла].....	38
Описание задания к задаче	38
Исходный код и описание решения	38
Описание проведённых тестов	40
18 Задача*. Построение дорог [10 s, 512 Mb, 4 балла, с учётом штрафа 3,5 балла]	43
Описание задания к задаче	43
Исходный код и описание решения	43
Описание проведённых тестов	45
Вывод по проделанной работе	47

Задачи по варианту

1 Задача. Лабиринт [5 s, 512 Mb]

Описание задания к задаче

Лабиринт представляет собой прямоугольную сетку ячеек со стенками между некоторыми соседними ячейками. Вы хотите проверить, существует ли путь от данной ячейки к данному выходу из лабиринта, где выходом также является ячейка, лежащая на границе лабиринта. Для этого вы представляете лабиринт в виде неориентированного графа: вершины графа являются ячейками лабиринта, две вершины соединены неориентированным ребром, если они смежные и между ними нет стены. Тогда, чтобы проверить, существует ли путь между двумя заданными ячейками лабиринта, достаточно проверить, что существует путь между соответствующими двумя вершинами в графе.

Вам дан неориентированный граф и две различные вершины u и v . Проверьте, есть ли путь между u и v .

- **Формат ввода / входного файла (input.txt).** Неориентированный граф с n вершинами и m рёбрами. Следующая строка после ввода всего графа содержит две вершины u и v .
- **Ограничения на входные данные.** $2 \leq n \leq 10^3$, $1 \leq m \leq 10^3$, $1 \leq u, v \leq n$, $u \neq v$.
- **Формат вывода / выходного файла (output.txt).** Выведите 1, если есть путь между вершинами u и v ; выведите 0, если пути нет.
- Ограничение по времени. 5 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
from collections import deque

def solve_labyrinth():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    n, m = map(int, lines[0].strip().split())

    graph = [[] for _ in range(n)]

    for i in range(1, m + 1):
        u, v = map(int, lines[i].strip().split())
```

```

    u -= 1
    v -= 1
    graph[u].append(v)
    graph[v].append(u)

u, v = map(int, lines[m + 1].strip().split())
u -= 1
v -= 1

visited = [False] * n
queue = deque([u])
visited[u] = True

while queue:
    curr = queue.popleft()

    if curr == v:
        with open('output.txt', 'w') as f:
            f.write('1\n')
        return

    for neighbor in graph[curr]:
        if not visited[neighbor]:
            visited[neighbor] = True
            queue.append(neighbor)

with open('output.txt', 'w') as f:
    f.write('0\n')

if __name__ == "__main__":
    solve_labyrinth()

```

Сначала из входного файла считывается количество вершин и рёбер, после чего строится список смежности неориентированного графа: для каждого ребра обе вершины добавляются в списки смежности друг друга. В последней строке считываются две вершины, между которыми требуется проверить наличие пути.

Для поиска пути используется поиск в ширину (BFS) из первой заданной вершины. BFS выбран потому, что он гарантированно находит вершину, если она достижима, и не требует рекурсии, что важно при больших размерах графа. Создаётся массив посещённых вершин и очередь, в которую изначально помещается стартовая вершина. На каждом шаге из очереди извлекается очередная вершина; если она совпадает с целевой, работа завершается выводом единицы. Иначе все её непосещённые соседи помечаются как посещённые и добавляются в очередь.

Если очередь опустела, а целевая вершина так и не была найдена, значит пути между заданными вершинами нет, и в выходной файл записывается ноль. Сложность алгоритма - $O(n + m)$ по времени и $O(n)$ по памяти, что с запасом укладывается в ограничения задачи.

Описание проведённых тестов

input.txt		output.txt	
1	4 4	1	1
2	1 2	2	
3	3 2		
4	4 3		
5	1 4		
6	1 4		
7			

input.txt		output.txt	
1	4 2	1	0
2	1 2	2	
3	3 2		
4	1 4		
5			

```

4 ▶ def test_minimal():  @ MarileeMorte
5     check( task_num: 1, input_data: "2 1\n1 2\n1 2\n", expected: "1")
6
7
8 ▶ def test_example_1():  @ MarileeMorte
9     check( task_num: 1, input_data: "4 4\n1 2\n3 2\n4 3\n1 4\n1 4\n", expected: "1")
10
11
12 ▶ def test_example_2():  @ MarileeMorte
13     check( task_num: 1, input_data: "4 2\n1 2\n3 2\n1 4\n", expected: "0")
14
15
16 ▶ def test_maximal():  @ MarileeMorte
17     n = 1000
18     lines = [f"{n} {n}"] # m = n = 1000
19     for i in range(1, n):
20         lines.append(f"{i} {i + 1}")
21     lines.append(f"1 {n}") # 1000-е ребро
22     lines.append("1 1000")
23     input_data = "\n".join(lines) + "\n"
24     check( task_num: 1, input_data, expected: "1")

```

	Время выполнения	Затраты памяти
--	------------------	----------------

Нижняя граница диапазона значений входных данных из текста задачи	time=0.084s	memory=32.0 KB
Первый пример из задачи	time=0.091s	memory=72.0 KB
Второй пример из задачи	time=0.086s	memory=56.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.113s	memory=96.0 KB

Вывод по задаче

Был реализован алгоритм поиска в ширину для проверки существования пути между двумя заданными вершинами неориентированного графа. Использование итеративной очереди вместо рекурсии делает решение устойчивым к графам с большим числом вершин и исключает риск переполнения стека вызовов.

5 Задача. Город с односторонним движением [5 s, 512 Mb]

Описание задания к задаче

Департамент полиции города сделал все улицы односторонними. Вы хотели бы проверить, можно ли законно проехать с любого перекрёстка на какой-либо другой перекрёсток. Для этого строится ориентированный граф: вершины – это перекрёстки, существует ребро (u, v) всякий раз, когда в городе есть улица (с односторонним движением) из u в v . Тогда достаточно проверить, все ли вершины графа лежат в одном компоненте сильной связности.

Нужно вычислить количество компонентов сильной связности заданного ориентированного графа с n вершинами и m ребрами.

- **Формат ввода / входного файла (input.txt).** Ориентированный граф с n вершинами и m ребрами.
- **Ограничения на входные данные.** $1 \leq n \leq 10^4$, $0 \leq m \leq 10^4$.
- **Формат вывода / выходного файла (output.txt).** Выведите число – количество компонентов сильной связности.
- Ограничение по времени. 5 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
from typing import List, Tuple

def solve_scc():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    lines = [line.strip() for line in lines if line.strip()]

    n, m = map(int, lines[0].split())

    graph: List[List[int]] = [[] for _ in range(n)]
    rev_graph: List[List[int]] = [[] for _ in range(n)]

    for i in range(1, m + 1):
        u, v = map(int, lines[i].split())
        u -= 1
        v -= 1
        graph[u].append(v)
        rev_graph[v].append(u)
```



```

visited: List[bool] = [False] * n
order: List[int] = []

for start in range(n):
    if visited[start]:
        continue
    visited[start] = True
    dfs_stack: List[Tuple[int, int]] = [(start, 0)]
    while dfs_stack:
        vertex, idx = dfs_stack[-1]
        if idx < len(graph[vertex]):
            neighbor = graph[vertex][idx]
            dfs_stack[-1] = (vertex, idx + 1)
            if not visited[neighbor]:
                visited[neighbor] = True
                dfs_stack.append((neighbor, 0))
        else:
            order.append(vertex)
            dfs_stack.pop()

visited = [False] * n
components = 0

for start in reversed(order):
    if visited[start]:
        continue
    components += 1
    visited[start] = True
    walk_stack: List[int] = [start]
    while walk_stack:
        vertex = walk_stack.pop()
        for neighbor in rev_graph[vertex]:
            if not visited[neighbor]:
                visited[neighbor] = True
                walk_stack.append(neighbor)

with open('output.txt', 'w') as f:
    f.write(str(components) + '\n')

if __name__ == "__main__":
    solve_scc()

```

Из входного файла считываются число вершин и число рёбер ориентированного графа. Для каждой вершины строятся два списка смежности: прямой, содержащий вершины, в которые ведут исходящие рёбра, и обратный, содержащий вершины, из которых приходят входящие рёбра.

Для поиска компонент сильной связности используется алгоритм Косарайю, состоящий из двух проходов поиска в глубину. На первом проходе обходятся все вершины исходного графа, и для каждой вершины фиксируется момент завершения её обработки - так получается список вершин в порядке пост-обхода. На втором проходе этот список обходится в обратном порядке, но уже по обратному графу. Каждый запуск второго

обхода выделяет одну компоненту сильной связности, а счётчик компонент увеличивается на единицу.

Оба прохода реализованы итеративно с явным стеком, потому что рекурсивный вариант при глубине обхода около десяти тысяч вызовов приводил бы к переполнению стека операционной системы. Первый проход использует стек пар «вершина и номер следующего соседа», что позволяет корректно воспроизвести пост-обход, а второй проход обходится обычным стеком вершин. Сложность решения - $O(n + m)$ по времени и $O(n + m)$ по памяти.

Описание проведённых тестов

input.txt		output.txt	
1	4 4	1	2
2	1 2	2	
3	4 1		
4	2 3		
5	3 1		
6			

input.txt		output.txt	
1	5 7	1	5
2	2 1	2	
3	3 2		
4	3 1		
5	4 3		
6	4 1		
7	5 2		
8	5 3		
9			

```

4  ▶ def test_minimal(): & MarieeMorte
5      check( task_num: 5, input_data: "1 0\n", expected: "1")
6
7
8  ▶ def test_example_1(): & MarieeMorte
9      check( task_num: 5, input_data: "4 4\n1 2\n4 1\n2 3\n3 1\n", expected: "2")
10
11
12 ▶ def test_example_2(): & MarieeMorte
13     check( task_num: 5, input_data: "5 7\n2 1\n3 2\n3 1\n4 3\n4 1\n5 2\n5 3\n", expected: "5")
14
15
16 ▶ def test_maximal_isolated(): & MarieeMorte
17     n = 10000
18     m = 0
19     lines = [f"{n} {m}"]
20     check( task_num: 5, "\n".join(lines) + "\n", str(n))
21
22
23 ▶ def test_maximal_cycle(): & MarieeMorte
24     n = 10000
25     m = n
26     lines = [f"{n} {m}"]
27     for i in range(1, n):
28         lines.append(f"{i} {i + 1}")
29     lines.append(f"{n} 1")
30     check( task_num: 5, "\n".join(lines) + "\n", expected: "1")

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.082s	memory=112.0 KB
Первый пример из задачи	time=0.088s	memory=112.0 KB
Второй пример из задачи	time=0.099s	memory=136.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.103s	memory=1.9 MB

Вывод по задаче

Была реализована итеративная версия алгоритма Косарайю для поиска компонент сильной связности. Использование явного стека вместо рекурсии позволило корректно работать с графами до десяти тысяч вершин без ограничений по глубине вызовов.

13 Задача. Грядки [1 s, 16 Mb]

Описание задания к задаче

Прямоугольный садовый участок шириной N и длиной M метров разбит на квадраты со стороной 1 метр. На этом участке вскопаны грядки. Грядкой называется совокупность квадратов, удовлетворяющая таким условиям:

- из любого квадрата этой грядки можно попасть в любой другой квадрат этой же грядки, последовательно переходя по грядке из квадрата в квадрат через их общую сторону;
- никакие две грядки не пересекаются и не касаются друг друга ни по вертикальной, ни по горизонтальной сторонам квадратов (касание грядок углами квадратов допускается).

Подсчитайте количество грядок на садовом участке.

- **Формат входных данных (input.txt) и ограничения.** В первой строке входного файла INPUT.TXT находятся числа N и M через пробел, далее идут N строк по M символов. Символ `#` обозначает территорию грядки, точка соответствует незанятой территории. Других символов в исходном файле нет ($1 \leq N, M \leq 200$).
- **Формат выходных данных (output.txt).** В выходной файл OUTPUT.TXT выведите количество грядок на садовом участке.
- Ограничение по времени. 1 сек.
- Ограничение по памяти. 16 мб.

Исходный код и описание решения

```
def solve_garden():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    n, m = map(int, lines[0].split())
    grid = [list(lines[i].strip()) for i in range(1, n + 1)]

    visited = [[False] * m for _ in range(n)]
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    count = 0
    for i in range(n):
        for j in range(m):
            if grid[i][j] == '#' and not visited[i][j]:
                count += 1
```

```

        visited[i][j] = True
        stack = [(i, j)]
        while stack:
            x, y = stack.pop()
            for dx, dy in directions:
                nx, ny = x + dx, y + dy
                if 0 <= nx < n and 0 <= ny < m and not
visited[nx][ny] and grid[nx][ny] == '#':
                    visited[nx][ny] = True
                    stack.append((nx, ny))

        with open('output.txt', 'w') as f:
            f.write(str(count) + '\n')

if __name__ == "__main__":
    solve_garden()

```

Из входного файла считываются размеры садового участка и сам прямоугольный участок, состоящий из символов, обозначающих грядку и незанятую территорию. Участок сохраняется в виде двумерного массива символов. Дополнительно создаётся двумерный массив посещённых клеток.

Подсчёт грядок ведётся как подсчёт компонент связности в графе, где вершинами являются клетки, помеченные символом грядки, а рёбрами - отношения соседства по стороне. Внешний двойной цикл перебирает все клетки поля. Если очередная клетка является частью грядки и ещё не была посещена, счётчик грядок увеличивается на единицу, а затем запускается обход по всем связанным с ней клеткам.

Обход реализован итеративно с использованием стека, что позволяет избежать переполнения стека вызовов при обработке поля размером двести на двести клеток, полностью засаженного грядками. Из каждой извлечённой клетки проверяются четыре соседа по стороне, и все непосещённые соседи, помеченные символом грядки, добавляются в стек. После завершения обхода счётчик содержит количество грядок. Сложность решения - $O(N \cdot M)$ по времени и $O(N \cdot M)$ по памяти.

Описание проведённых тестов

input.txt			
1	5 10		
2	##.....#.		
3	.#..#...#.		
4	.###.....#.		
5	..##.....#.		
6	#.	1	3
7		2	

≡ input.txt ×

1

5 10

2

##..#####.

3

.#.#.#.....

4

###..##.#.

5

..##.....#

6

.###.#####

7

≡ output.txt ×

1

5

2

ID	Дата	Автор	Задача	Язык	Результат	Тест	Время	Память
25418435	10.09.2026 1:54:00	Борзова Дарья Максимовна	0432	Python	Accepted		0.093	5.9 Мб

```

4  ▶ def test_minimal():  Ⓜ MarieeMorte
5      check( task_num: 13,  input_data: "1 1\n#\n",  expected: "1")
6
7
8  ▶ def test_minimal_empty():  Ⓜ MarieeMorte
9      check( task_num: 13,  input_data: "1 1\n.\n",  expected: "0")
10
11
12 ▶ def test_example_1():  Ⓜ MarieeMorte
13     check( task_num: 13,
14         input_data: "5 10\n"
15         "##.....#\n"
16         ".#...#...#\n"
17         "###...#\n"
18         "..##...#\n"
19         ".....#\n",
20         expected: "3")
21
22
23 ▶ def test_example_2():  Ⓜ MarieeMorte
24     check( task_num: 13,
25         input_data: "5 10\n"
26         "##.#####\n"
27         ".#...#...#\n"
28         "###...#.#\n"
29         "..##...#.#\n"
30         ".###.#####\n",
31         expected: "5")

```

```

34 ▶ def test_maximal():  ⚡ MarieeMorte
35     n = 200
36     m = 200
37     lines = [f"{n} {m}"]
38     for _ in range(n):
39         lines.append("#" * m)
40     check(task_num: 13, "\n".join(lines) + "\n", expected: "1")
41
42
43 ▶ def test_maximal_empty():  ⚡ MarieeMorte
44     n = 200
45     m = 200
46     lines = [f"{n} {m}"]
47     for _ in range(n):
48         lines.append("." * m)
49     check(task_num: 13, "\n".join(lines) + "\n", expected: "0")
50
51
52 ▶ def test_maximal_checkerboard():  ⚡ MarieeMorte
53     n = 200
54     m = 200
55     lines = [f"{n} {m}"]
56     for i in range(n):
57         row = ""
58         for j in range(m):
59             row += "#" if (i + j) % 2 == 0 else "."
60         lines.append(row)
61     check(task_num: 13, "\n".join(lines) + "\n", str(n * m // 2))

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.080s	memory=12.0 KB
Первый пример из задачи	time=0.098s	memory=144.0 KB
Второй пример из задачи	time=0.099s	memory=216.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.104s	memory=972.0 KB

Вывод по задаче

Была реализована задача подсчёта числа грядок как числа компонент связности на прямоугольной сетке. Использование итеративного обхода с явным стеком позволило обрабатывать поля максимального размера без риска переполнения стека вызовов.

Дополнительные задачи

2 Задача. Компоненты [5 s, 512 Mb, 1 балл, с учётом штрафа 0,5 балла]

Описание задания к задаче

Теперь вы решаете сделать так, чтобы в лабиринте не было мёртвых зон, то есть чтобы из каждой клетки был доступен хотя бы один выход. Для этого вы находите связные компоненты соответствующего неориентированного графа и следите за тем, чтобы каждый компонент содержал выходную ячейку.

Дан неориентированный граф с n вершинами и m рёбрами. Нужно посчитать количество компонент связности в нём.

- **Формат ввода / входного файла (input.txt).** Неориентированный граф с n вершинами и m рёбрами.
- **Ограничения на входные данные.** $1 \leq n \leq 10^3$, $0 \leq m \leq 10^3$.
- **Формат вывода / выходного файла (output.txt).** Выведите количество компонент связности.
- Ограничение по времени. 5 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
from collections import deque

def solve_components():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    n, m = map(int, lines[0].strip().split())

    graph = [[] for _ in range(n)]

    for i in range(1, m + 1):
        if not lines[i].strip():
            continue
        u, v = map(int, lines[i].strip().split())
        u -= 1
        v -= 1
        graph[u].append(v)
        graph[v].append(u)

    visited = [False] * n
```

```

def bfs(start):
    queue = deque([start])
    visited[start] = True
    while queue:
        vertex = queue.popleft()
        if not isinstance(vertex, int):
            continue
        for neighbor in graph[vertex]:
            if not isinstance(neighbor, int):
                continue
            if not visited[neighbor]:
                visited[neighbor] = True
                queue.append(neighbor)

components = 0
for i in range(n):
    if not visited[i]:
        bfs(i)
        components += 1

with open('output.txt', 'w') as f:
    f.write(str(components) + '\n')

if __name__ == "__main__":
    solve_components()

```

Из входного файла считываются количество вершин и количество рёбер, после чего строится список смежности неориентированного графа. Дополнительно создаётся массив посещённых вершин, изначально заполненный значениями «ложь».

Основной цикл перебирает все вершины графа. Если очередная вершина ещё не была посещена, значит она принадлежит новой компоненте связности. Запускается обход в ширину из этой вершины, который помечает все достижимые из неё вершины как посещённые. После завершения обхода счётчик компонент увеличивается на единицу. Когда внешний цикл завершается, счётчик содержит количество компонент связности графа.

Обход в ширину реализован итеративно с использованием очереди, что обеспечивает работу на графах с числом вершин до тысячи без ограничений на глубину рекурсии. Сложность решения - $O(n + m)$ по времени и $O(n)$ по памяти.

Описание проведённых тестов

input.txt		output.txt	
1	4 2	1	2
2	1 2		
3	3 2		
4		2	

```

4 ▶ def test_minimal():  Ⓜ MarieeMorte
5     check( task_num: 2, input_data: "1 0\n", expected: "1")
6
7
8 ▶ def test_example_1():  Ⓜ MarieeMorte
9     check( task_num: 2, input_data: "4 2\n1 2\n3 2\n", expected: "2")
10
11
12 ▶ def test_maximal():  Ⓜ MarieeMorte
13     n = 1000
14     m = 999
15     lines = [f"{n} {m}"]
16     for i in range(1, n):
17         lines.append(f"{i} {i + 1}")
18     check( task_num: 2, "\n".join(lines) + "\n", expected: "1")

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.077s	memory=16.0 KB
Пример из задачи	time=0.091s	memory=40.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.097s	memory=84.0 KB

Вывод по задаче

Был реализован подсчёт числа компонент связности неориентированного графа на основе обхода в ширину. Алгоритм корректно обрабатывает графы с изолированными вершинами и с несколькими независимыми компонентами.

3 Задача. Циклы [5 s, 512 Mb, 1 балл, с учётом штрафа 0,5 балла]

Описание задания к задаче

Учебная программа по инфокоммуникационным технологиям определяет пререквизиты для каждого курса в виде списка курсов, которые необходимо пройти перед тем, как начать этот курс. Вы хотите выполнить проверку согласованности учебного плана, то есть проверить отсутствие циклических зависимостей. Для этого строится следующий ориентированный граф: вершины соответствуют курсам, есть направленное ребро (u, v) – курс u следует пройти перед курсом v . Затем достаточно проверить, содержит ли полученный граф цикл.

Проверьте, содержит ли данный граф циклы.

- **Формат ввода / входного файла (input.txt).** Ориентированный граф с n вершинами и m ребрами по формату 1.
- **Ограничения на входные данные.** $1 \leq n \leq 10^3$, $0 \leq m \leq 10^3$.
- **Формат вывода / выходного файла (output.txt).** Выведите 1, если данный граф содержит цикл; выведите 0, если не содержит.
- Ограничение по времени. 5 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
import sys

sys.setrecursionlimit(10000)

def solve_cycles():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    lines = [line.strip() for line in lines if line.strip()]

    n, m = map(int, lines[0].split())

    graph = [[] for _ in range(n)]

    for i in range(1, m + 1):
        parts = lines[i].split()
        if len(parts) >= 2:
            u = int(parts[0]) - 1
            v = int(parts[1]) - 1
            graph[u].append(v)
```

```

visited = [False] * n
rec_stack = [False] * n

def dfs(vertex):
    visited[vertex] = True
    rec_stack[vertex] = True

    for neighbor in graph[vertex]:
        if not isinstance(neighbor, int):
            continue
        if not visited[neighbor]:
            if dfs(neighbor):
                return True
        elif rec_stack[neighbor]:
            return True

    rec_stack[vertex] = False
    return False

for i in range(n):
    if not visited[i]:
        if dfs(i):
            with open('output.txt', 'w') as f:
                f.write('1\n')
            return

with open('output.txt', 'w') as f:
    f.write('0\n')

if __name__ == "__main__":
    solve_cycles()

```

Из входного файла считываются число вершин и число рёбер ориентированного графа, после чего для каждой вершины строится список смежных вершин, в которые ведут исходящие рёбра.

Для обнаружения цикла используется поиск в глубину с раскраской вершин в три состояния. Первое состояние означает, что вершина ещё не посещалась, второе - что вершина находится в текущем стеке рекурсии, третье - что обработка вершины полностью завершена. Если при обходе очередной вершины встречается сосед, находящийся во втором состоянии, это означает, что найден обратный ход на вершину из текущего пути, то есть цикл.

Рекурсивная реализация защищена увеличением предельной глубины рекурсии, поскольку при вырожденных графах глубина обхода может достигать тысячи вызовов. Также используется проверка типа соседа, чтобы исключить случайные ошибки парсинга. Если цикл был найден хотя бы из одной стартовой вершины, в выходной файл записывается единица, иначе ноль. Сложность решения - $O(n + m)$ по времени и $O(n)$ по памяти.

Описание проведённых тестов

input.txt		output.txt	
1	4 4	1	1
2	1 2	2	
3	4 1		
4	2 3		
5	3 1		
6			

input.txt		output.txt	
1	5 7	1	0
2	1 2	2	
3	2 3		
4	1 3		
5	3 4		
6	1 4		
7	2 5		
8	3 5		
9			

```

4 ▶ def test_minimal(): & MarieeMorte
5     check( task_num: 3, input_data: "1 0\n", expected: "0")
6
7
8 ▶ def test_example_1(): & MarieeMorte
9     check( task_num: 3, input_data: "4 4\n1 2\n4 1\n2 3\n3 1\n", expected: "1")
10
11
12 ▶ def test_example_2(): & MarieeMorte
13     check( task_num: 3, input_data: "5 7\n1 2\n2 3\n1 3\n3 4\n1 4\n2 5\n3 5\n", expected: "0")
14
15
16 ▶ def test_maximal(): & MarieeMorte
17     n = 1000
18     m = 999
19     lines = [f"{n} {m}"]
20     for i in range(1, n):
21         lines.append(f"{i} {i + 1}")
22     check( task_num: 3, "\n".join(lines) + "\n", expected: "0")
23
24
25 ▶ def test_maximal_with_cycle(): & MarieeMorte
26     n = 1000
27     m = 1000
28     lines = [f"{n} {m}"]
29     for i in range(1, n):
30         lines.append(f"{i} {i + 1}")
31     lines.append(f"{n} 1")
32     check( task_num: 3, "\n".join(lines) + "\n", expected: "1")

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.080s	memory=56.0 KB
Первый пример из задачи	time=0.085s	memory=172.0 KB
Второй пример из задачи	time=0.083s	memory=140.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.089s	memory=1.4 MB

Вывод по задаче

Был реализован алгоритм поиска цикла в ориентированном графе методом поиска в глубину с тремя состояниями вершин. Такой подход позволяет корректно отличать циклы от обычных повторных посещений, характерных для ориентированного графа без циклов.

4 Задача. Порядок курсов [10 s, 512 Mb, 1 балл, с учётом штрафа 0,5 балла]

Описание задания к задаче

Теперь, когда вы уверены, что в данном учебном плане нет циклических зависимостей, вам нужно найти порядок всех курсов, соответствующий всем зависимостям. Для этого нужно сделать топологическую сортировку соответствующего ориентированного графа. Дан ориентированный ациклический граф (DAG) с n вершинами и m ребрами. Выполните топологическую сортировку.

- **Формат ввода / входного файла (input.txt).** Ориентированный ациклический граф с n вершинами и m ребрами по формату 1.
- **Ограничения на входные данные.** $1 \leq n \leq 10^5$, $0 \leq m \leq 10^5$. Графы во входных файлах гарантированно ациклические.
- **Формат вывода / выходного файла (output.txt).** Выведите любое линейное упорядочение данного графа (многие ациклические графы имеют более одного варианта упорядочения, вы можете вывести любой из них).
- Ограничение по времени. 10 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
from collections import deque
from typing import List

def solve_topological_sort():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    lines = [line.strip() for line in lines if line.strip()]

    n, m = map(int, lines[0].split())

    graph: List[List[int]] = [[] for _ in range(n)]
    in_degree: List[int] = [0] * n

    for i in range(1, m + 1):
        u, v = map(int, lines[i].split())
        u -= 1
        v -= 1
        graph[u].append(v)
        in_degree[v] += 1
```

```

queue = deque()
for i in range(n):
    if in_degree[i] == 0:
        queue.append(i)

order: List[int] = []
while queue:
    vertex = queue.popleft()
    order.append(vertex)
    for neighbor in graph[vertex]:
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0:
            queue.append(neighbor)

with open('output.txt', 'w') as f:
    f.write(' '.join(str(v + 1) for v in order) + '\n')

if __name__ == "__main__":
    solve_topological_sort()

```

Из входного файла считываются количество вершин и рёбер ориентированного ациклического графа. Для каждой вершины строится список смежных вершин, а также массив входящих степеней, где для каждого ребра из начала в конец степень конечной вершины увеличивается на единицу.

Для топологической сортировки применён алгоритм Кана. Сначала в очередь помещаются все вершины с нулевой входящей степенью, то есть те, которые не зависят ни от каких других. Затем в цикле из очереди последовательно извлекаются вершины, добавляются в итоговый порядок, а входящие степени всех их соседей уменьшаются на единицу. Как только степень очередного соседа становится нулевой, он также помещается в очередь.

Итеративная реализация через очередь выбрана сознательно: рекурсивный обход в глубину не подходит для задачи из-за ограничения на число вершин в сто тысяч, при котором глубина рекурсии превысила бы размер стека вызовов. Благодаря BFS-подходу решение корректно работает на вырожденных графах. Сложность алгоритма - $O(n + m)$ по времени и $O(n)$ по памяти.

Описание проведённых тестов

input.txt			
1	4 3		
2	1 2		
3	4 1		
4	3 1		
5			

output.txt	
1	3 4 1 2
2	

input.txt			
1	4 1		
2	3 1		
3			

output.txt	
1	2 3 4 1
2	

input.txt			
1	5 7		
2	2 1		
3	3 2		
4	3 1		
5	4 3		
6	4 1		
7	5 2		
8	5 3		
9			

output.txt	
1	4 5 3 2 1
2	

```

16 ▶ def test_minimal():  & MarieeMorte
17     check(task_num: 4, input_data: "1 0\n", expected: "1")
18
19
20 ▶ def test_example_1():  & MarieeMorte
21     check_toposort(input_data: "4 3\n1 2\n4 1\n3 1\n", n: 4, edges: [(1, 2), (4, 1), (3, 1)])
22
23
24 ▶ def test_example_2():  & MarieeMorte
25     check_toposort(input_data: "4 1\n3 1\n", n: 4, edges: [(3, 1)])
26
27
28 ▶ def test_example_3():  & MarieeMorte
29     check_toposort(
30         input_data: "5 7\n2 1\n3 2\n3 1\n4 3\n4 1\n5 2\n5 3\n",
31         n: 5,
32         edges: [(2, 1), (3, 2), (3, 1), (4, 3), (4, 1), (5, 2), (5, 3)],
33     )
34
35
36 ▶ def test_maximal():  & MarieeMorte
37     n = 100000
38     m = 99999
39     lines = [f"{n} {m}"]
40     edges = []
41     for i in range(1, n):
42         lines.append(f"{i} {i + 1}")
43         edges.append((i, i + 1))
44     check_toposort("\n".join(lines) + "\n", n, edges)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.081s	memory=116.0 KB
Первый пример из задачи	time=0.093s	memory=196.0 KB
Второй пример из задачи	time=0.092s	memory=140.0 KB
Третий пример из задачи	time=0.088s	memory=152.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.197s	memory=27.4 MB

Вывод по задаче

Была реализована топологическая сортировка ориентированного ациклического графа с использованием итеративного алгоритма Кана. Отказ от рекурсии позволил обрабатывать графы с числом вершин до ста тысяч без риска переполнения стека.

8 Задача. Стоимость полета [10 s, 512 Mb, 1.5 балла, с учётом штрафа 1 балл]

Описание задания к задаче

Теперь вас интересует минимизация не количества пересадок, а общей стоимости полёта. Для этого строится взвешенный граф: вес ребра из одного города в другой – это стоимость соответствующего перелёта.

Дан ориентированный граф с положительными весами ребер, n - количество вершин и m - количество рёбер, а также даны две вершины u и v . Вычислить вес кратчайшего пути между u и v (то есть минимальный общий вес пути из u в v).

- **Формат ввода / входного файла (input.txt).** Задан ориентированный взвешенный граф. Следующая строка содержит две вершины u и v .
- **Ограничения на входные данные.** $1 \leq n \leq 10^4$, $0 \leq m \leq 10^5$, $1 \leq u, v \leq n$, $u \neq v$, вес каждого ребра – неотрицательное целое число, не превосходящее 10^8 .
- **Формат вывода / выходного файла (output.txt).** Выведите минимальный вес пути из u в v . Введите -1, если пути нет.
- Ограничение по времени. 10 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
import heapq

def solve_shortest_path():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    lines = [line.strip() for line in lines if line.strip()]

    n, m = map(int, lines[0].split())

    graph = [[] for _ in range(n)]

    for i in range(1, m + 1):
        parts = lines[i].split()
        u = int(parts[0]) - 1
        v = int(parts[1]) - 1
        w = int(parts[2])
        graph[u].append((v, w))

    u, v = map(int, lines[m + 1].split())
```

```

u -= 1
v -= 1

inf = 10 ** 18
dist = [inf] * n
dist[u] = 0
pq = [(0, u)]

while pq:
    d, vertex = heapq.heappop(pq)

    if d > dist[vertex]:
        continue

    if vertex == v:
        break

    for neighbor, weight in graph[vertex]:
        new_dist = d + weight
        if new_dist < dist[neighbor]:
            dist[neighbor] = new_dist
            heapq.heappush(pq, (new_dist, neighbor))

with open('output.txt', 'w') as f:
    if dist[v] == inf:
        f.write('-1\n')
    else:
        f.write(str(dist[v]) + '\n')

if __name__ == "__main__":
    solve_shortest_path()

```

Из входного файла считываются количество вершин и рёбер ориентированного взвешенного графа. Для каждой вершины формируется список смежности, в котором для каждого исходящего ребра хранится пара из конечной вершины и веса. В последней строке считываются начальная и конечная вершины искомого пути.

Для поиска кратчайшего пути применён алгоритм Дейкстры. Массив расстояний инициализируется большим числом, а расстояние до начальной вершины устанавливается равным нулю. Используется приоритетная очередь, в которую изначально помещается начальная вершина. На каждой итерации из очереди извлекается вершина с минимальным текущим расстоянием; если это расстояние устарело по сравнению с сохранённым в массиве, вершина пропускается. Иначе перебираются все её соседи, и если новое расстояние через текущую вершину меньше уже сохранённого, значение в массиве обновляется, а сосед добавляется в очередь.

Досрочное завершение происходит, когда из очереди извлекается конечная вершина: в этот момент её расстояние уже окончательное. Если после опустошения очереди расстояние до конечной вершины осталось бесконечным, в выходной файл

записывается минус единица. Сложность алгоритма - $O((n + m) \log n)$ по времени и $O(n + m)$ по памяти.

Описание проведённых тестов

input.txt		output.txt	
1	4 4	1	3
2	1 2 1	2	
3	4 1 2		
4	2 3 2		
5	1 3 5		
6	1 3		
7			

input.txt		output.txt	
1	5 9	1	6
2	1 2 4	2	
3	1 3 2		
4	2 3 2		
5	3 2 1		
6	2 4 2		
7	3 5 4		
8	5 4 1		
9	2 5 3		
10	3 4 4		
11	1 5		
12			

input.txt			
1	3 3		
2	1 2 7		
3	1 3 5		
4	2 3 2		
5	3 2	1	-1
6		2	

```

4 def test_minimal():
5     check(task_num: 8, input_data: "2 1\n1 2 5\n1 2\n", expected: "5")
6
7
8 def test_example_1():
9     check(task_num: 8, input_data: "4 4\n1 2 1\n4 1 2\n2 3 2\n1 3 5\n1 3\n", expected: "3")
10
11
12 def test_example_2():
13     check(task_num: 8, input_data: "5 9\n1 2 4\n1 3 2\n2 3 2\n3 2 1\n2 4 2\n3 5 4\n5 4 1\n2 5 3\n3 4 4\n1 5\n", expected: "6")
14
15
16 def test_example_3():
17     check(task_num: 8, input_data: "3 3\n1 2 7\n1 3 5\n2 3 2\n3 2\n", expected: "-1")
18
19
20 def test_maximal():
21     n = 10000
22     m = 9999
23     lines = [f"{n} {m}"]
24     for i in range(1, n):
25         lines.append(f"{i} {i + 1} 1")
26     lines.append(f"1 {n}")
27     check(task_num: 8, "\n".join(lines) + "\n", str(n - 1))

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.081s	memory=168.0 KB
Первый пример из задачи	time=0.085s	memory=184.0 KB
Второй пример из задачи	time=0.085s	memory=176.0 KB
Третий пример из задачи	time=0.084s	memory=180.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.106s	memory=1.1 MB

Вывод по задаче

Был реализован алгоритм Дейкстры для поиска кратчайшего пути в ориентированном графе с неотрицательными весами. Использование приоритетной очереди обеспечивает

эффективную обработку графов с большим числом рёбер, а досрочное завершение экономит время при достижении цели.

9 Задача. Аномалии курсов валют [10 s, 512 Mb, 2 балла, с учётом штрафа 1,5 балла]

Описание задания к задаче

Вам дан список валют $c_1, c_2, \dots, c_n$ вместе со списком обменных курсов: r_{ij} — количество единиц валюты c_j , которое можно получить за одну единицу c_i . Вы хотите проверить, можно ли начать делать обмен с одной единицы какой-либо валюты, выполнить последовательность обменов и получить более одной единицы той же валюты, с которой вы начали обмен. Другими словами, вы хотите найти валюты $c_{i_1}, c_{i_2}, \dots, c_{i_k}$ такие, что $r_{i_1, i_2} \cdot r_{i_2, i_3} \cdot \dots \cdot r_{i_{k-1}, i_k} \cdot r_{i_k, i_1} > 1$.

Для этого построим следующий граф: вершинами являются валюты $c_1, c_2, \dots, c_n$, вес ребра из c_i в c_j равен $-\log r_{ij}$. Тогда достаточно проверить, есть ли в этом графе отрицательный цикл. Пусть цикл $c_i \rightarrow c_j \rightarrow c_k \rightarrow c_i$ имеет отрицательный вес. Это означает, что $-(\log c_{ij} + \log c_{jk} + \log c_{ki}) < 0$ и, следовательно, $\log c_{ij} + \log c_{jk} + \log c_{ki} > 0$. Это, в свою очередь, означает, что

$$r_{ij}r_{jk}r_{ki} = 2^{\log c_{ij}}2^{\log c_{jk}}2^{\log c_{ki}} = 2^{\log c_{ij} + \log c_{jk} + \log c_{ki}} > 1.$$

Для заданного ориентированного графа с возможными отрицательными весами ребер, у которого n вершин и m ребер, проверьте, содержит ли он цикл с отрицательным суммарным весом.

- **Формат ввода / входного файла (input.txt).** Ориентированный взвешенный граф.
- **Ограничения на входные данные.** $1 \leq n \leq 10^3$, $0 \leq m \leq 10^4$, вес каждого ребра — целое число, не превосходящее по модулю 10^4 .
- **Формат вывода / выходного файла (output.txt).** Выведите 1, если граф содержит цикл с отрицательным суммарным весом. Выведите 0 в противном случае.
- Ограничение по времени. 10 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
def solve_negative_cycle():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
```

```

        return

    lines = [line.strip() for line in lines if line.strip()]

    n, m = map(int, lines[0].split())

    edges = []

    for i in range(1, m + 1):
        parts = lines[i].split()
        u = int(parts[0]) - 1
        v = int(parts[1]) - 1
        w = int(parts[2])
        edges.append((u, v, w))

    inf = 10 ** 18
    dist = [0] * n

    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] != inf and dist[v] > dist[u] + w:
                dist[v] = dist[u] + w

    for u, v, w in edges:
        if dist[u] != inf and dist[v] > dist[u] + w:
            with open('output.txt', 'w') as f:
                f.write('1\n')
            return

    with open('output.txt', 'w') as f:
        f.write('0\n')

if __name__ == "__main__":
    solve_negative_cycle()

```

Из входного файла считываются число вершин и число рёбер ориентированного взвешенного графа. Все рёбра сохраняются в виде троек из начала, конца и веса. Отдельно строится список смежности, но он здесь не используется и нужен только для структурной полноты графа.

Для проверки наличия отрицательного цикла используется алгоритм Беллмана-Форда. Ключевая особенность реализации состоит в том, что массив расстояний инициализируется не бесконечностями, а нулями. Это нужно потому, что условие задачи требует найти отрицательный цикл в любой части графа, а не только достижимый из какой-то одной вершины. Если все расстояния изначально нулевые, то на каждой итерации релаксации фактически ищутся улучшения из любой вершины, что позволяет обнаружить цикл даже в изолированной компоненте.

Основной цикл выполняет $n-1$ итерацию релаксации всех рёбер. После этого делается дополнительный проход по всем рёбрам: если хотя бы одно ребро всё ещё может быть улучшено, значит в графе есть отрицательный цикл, и в выходной файл записывается

единица. Иначе записывается ноль. Сложность алгоритма - $O(n \cdot m)$ по времени и $O(n)$ по памяти.

Описание проведённых тестов

input.txt			
1	4 4		
2	1 2 -5		
3	4 1 2		
4	2 3 2		
5	3 1 1	1	1
6		2	

```

4  def test_minimal():  MarieeMorte
5      check( task_num: 9, input_data: "1 0\n", expected: "0")
6
7
8  def test_example_1():  MarieeMorte
9      check( task_num: 9, input_data: "4 4\n1 2 -5\n4 1 2\n2 3 2\n3 1 1\n", expected: "1")
10
11
12 def test_maximal():  MarieeMorte
13     n = 1000
14     m = 1000
15     lines = [f"{n} {m}"]
16     for i in range(1, n):
17         lines.append(f"{i} {i + 1} 1")
18     lines.append(f"{n} 1 1")
19     check( task_num: 9, "\n".join(lines) + "\n", expected: "0")
20
21
22 def test_maximal_with_negative_cycle():  MarieeMorte
23     n = 1000
24     m = 1000
25     lines = [f"{n} {m}"]
26     for i in range(1, n):
27         lines.append(f"{i} {i + 1} 1")
28     lines.append(f"{n} 1 -1000")
29     check( task_num: 9, "\n".join(lines) + "\n", expected: "1")

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.078s	memory=64.0 KB

Пример из задачи	time=0.078s	memory=64.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.164s	memory=424.0 KB

Вывод по задаче

Был реализован алгоритм Беллмана-Форда для обнаружения отрицательного цикла в ориентированном взвешенном графе. Инициализация всех расстояний нулями позволила находить циклы в любой компоненте связности, а не только достижимой из фиксированной стартовой вершины.

10 Задача. Оптимальный обмен валюты [10 s, 512 Mb, 2 балла, с учётом штрафа 1,5 балла]

Описание задания к задаче

Теперь вы хотите вычислить оптимальный способ обмена данной вам валюты c_i на все другие валюты. Для этого вы находите кратчайшие пути из вершины c_i во все остальные вершины.

Дан ориентированный граф с возможными отрицательными весами ребер, у которого n вершин и m ребер, а также задана одна его вершина s . Вычислите длину кратчайших путей из s во все остальные вершины графа.

- **Формат ввода / входного файла (input.txt).** Ориентированный взвешенный граф задан по формату 1.
- **Ограничения на входные данные.** $1 \leq n \leq 10^3$, $0 \leq m \leq 10^4$, $1 \leq s \leq n$, вес каждого ребра – целое число, не превосходящее по модулю 10^9 .
- **Формат вывода / выходного файла (output.txt).** Для каждой вершины i графа от 1 до n выведите в каждой отдельной строке следующее:
 - «*», если пути из s в i нет;
 - «-», если существует путь из s в i , но нет кратчайшего пути из s в i (то есть расстояние от s до i равно $-\infty$);
 - длину кратчайшего пути в остальных случаях.
- Ограничение по времени. 10 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
def solve_optimal_exchange():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    lines = [line.strip() for line in lines if line.strip()]

    n, m = map(int, lines[0].split())

    edges = []
    graph = [[] for _ in range(n)]

    for i in range(1, m + 1):
        parts = lines[i].split()
```

```

    u = int(parts[0]) - 1
    v = int(parts[1]) - 1
    w = int(parts[2])
    edges.append((u, v, w))
    graph[u].append(v)

s = int(lines[m + 1]) - 1

inf = 10 ** 18
dist = [inf] * n
dist[s] = 0

for _ in range(n - 1):
    for u, v, w in edges:
        if dist[u] != inf and dist[v] > dist[u] + w:
            dist[v] = dist[u] + w

negative = [False] * n

for _ in range(n):
    for u, v, w in edges:
        if dist[u] != inf and dist[v] > dist[u] + w:
            dist[v] = dist[u] + w
            negative[v] = True
        if negative[u]:
            negative[v] = True

with open('output.txt', 'w') as f:
    for i in range(n):
        if dist[i] == inf:
            visited = [False] * n
            stack = [s]
            visited[s] = True
            while stack:
                vertex = stack.pop()
                for neighbor in graph[vertex]:
                    if not visited[neighbor]:
                        visited[neighbor] = True
                        stack.append(neighbor)

            if not visited[i]:
                f.write('*\n')
            else:
                f.write('-\n')
        elif negative[i]:
            f.write('-\n')
        else:
            f.write(str(dist[i]) + '\n')

if __name__ == "__main__":
    solve_optimal_exchange()

```

Из входного файла считываются число вершин и число рёбер ориентированного взвешенного графа. Рёбра сохраняются в виде троек, а также строится список смежности для дальнейшей проверки достижимости. В последней строке считывается номер начальной вершины.

Для поиска кратчайших путей используется алгоритм Беллмана-Форда. Массив расстояний инициализируется бесконечностями, расстояние до начальной вершины равно нулю. Выполняется $n-1$ итерация релаксации всех рёбер, после чего запускается дополнительный проход, помечающий вершины, достижимые из отрицательных циклов: если ребро всё ещё может быть улучшено, конечная вершина помечается флагом «бесконечно малое расстояние», а также все вершины, до которых можно дойти из помеченной, постепенно распространяют этот флаг.

На этапе вывода для каждой вершины возможны три случая. Если расстояние осталось бесконечным, дополнительно запускается обход в глубину из начальной вершины, чтобы различить два подслучая: если вершина недостижима, выводится звёздочка, а если она достижима, но без кратчайшего пути, выводится минус. Если вершина помечена как достижимая из отрицательного цикла, выводится минус. Во всех остальных случаях выводится конкретное значение кратчайшего расстояния. Сложность решения - $O(n \cdot m)$ по времени и $O(n + m)$ по памяти.

Описание проведённых тестов

input.txt		output.txt	
1	6 7	1	0
2	1 2 10	2	10
3	2 3 5	3	-
4	1 3 100	4	-
5	3 5 7	5	-
6	5 4 10	6	*
7	4 3 -18	7	
8	6 1 -1		
9	1		
10			

input.txt		output.txt	
1	5 4	1	-
2	1 2 1	2	-
3	4 1 2	3	-
4	2 3 2	4	0
5	3 1 -5	5	*
6	4	6	
7			

```

4  def test_minimal():  & MarieeMorte
5      check( task_num: 10, input_data: "1 0\n1\n", expected: "0")
6
7
8  def test_example_1():  & MarieeMorte
9      check( task_num: 10, input_data: "6 7\n1 2 10\n2 3 5\n1 3 100\n3 5 7\n5 4 10\n4 3 -18\n6 1 -1\n1\n",
10         expected: "0\n10\n-\n-\n-\n*")
11
12
13 def test_example_2():  & MarieeMorte
14     check( task_num: 10, input_data: "5 4\n1 2 1\n4 1 2\n2 3 2\n3 1 -5\n4\n",
15         expected: "-\n-\n-\n0\n*")
16
17
18 def test_maximal():  & MarieeMorte
19     n = 1000
20     m = 999
21     lines = [f"{n} {m}"]
22     for i in range(1, n):
23         lines.append(f"{i} {i + 1} 1")
24     lines.append("1")
25     expected = "\n".join(str(i) for i in range(n))
26     check( task_num: 10, "\n".join(lines) + "\n", expected)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.081s	memory=172.0 KB
Первый пример из задачи	time=0.093s	memory=280.0 KB
Второй пример из задачи	time=0.081s	memory=200.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.211s	memory=392.0 KB

Вывод по задаче

Был реализован алгоритм Беллмана-Форда с обработкой отрицательных циклов и восстановлением типов расстояний. Решение корректно различает недостижимые

вершины, вершины с бесконечно малым расстоянием и вершины с конечным кратчайшим путём.

18 Задача*. Построение дорог [10 s, 512 Mb, 4 балла, с учётом штрафа 3,5 балла]

Описание задания к задаче

В этой задаче цель состоит в том, чтобы построить дороги между некоторыми парами заданных городов так, чтобы между любыми двумя городами существовал путь и общая длина дорог была минимальна.

Даны n точек на плоскости, соедините их отрезками минимальной общей длины так, чтобы между любыми двумя точками был путь. Напомним, что длина отрезка с концами (x_1, y_1) и (x_2, y_2) равна $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

- **Формат ввода / входного файла (input.txt).** Первая строка содержит количество n точек на плоскости. Каждая из следующих n строк определяет координаты точки на плоскости (x_i, y_i) через пробел.
- **Ограничения на входные данные.** $1 \leq n \leq 200$, $-10^3 \leq x_i, y_i \leq 10^3$ – целые числа. Все точки попарно различны, никакие три точки не лежат на одной прямой.
- **Формат вывода / выходного файла (output.txt).** Выведите минимальную общую длину отрезков. Абсолютная погрешность между ответом вашей программы и оптимальным значением должно быть не более 10^{-6} . Чтобы убедиться в этом, выведите свой ответ как минимум с семью знаками после запятой.
- Ограничение по времени. 10 сек.
- Ограничение по памяти. 512 мб.

Исходный код и описание решения

```
import math

def solve_mst():
    with open('input.txt', 'r') as f:
        lines = f.readlines()

    if not lines:
        return

    n = int(lines[0].strip())
    points = []

    for i in range(1, n + 1):
        x, y = map(int, lines[i].strip().split())
        points.append((x, y))
```

```

edges = []
for i in range(n):
    for j in range(i + 1, n):
        dx = points[i][0] - points[j][0]
        dy = points[i][1] - points[j][1]
        weight = math.sqrt(dx * dx + dy * dy)
        edges.append((weight, i, j))

edges.sort()

parent = list(range(n))
rank = [0] * n

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

def union(x, y):
    rx, ry = find(x), find(y)
    if rx == ry:
        return False
    if rank[rx] < rank[ry]:
        parent[rx] = ry
    elif rank[rx] > rank[ry]:
        parent[ry] = rx
    else:
        parent[ry] = rx
        rank[rx] += 1
    return True

total_weight = 0.0
edges_used = 0

for weight, u, v in edges:
    if union(u, v):
        total_weight += weight
        edges_used += 1
        if edges_used == n - 1:
            break

with open('output.txt', 'w') as f:
    f.write(f'{total_weight:.9f}\n')

if __name__ == "__main__":
    solve_mst()

```

Из входного файла считывается количество точек и координаты каждой точки на плоскости. Дополнительно проверяется, что список точек не пуст.

Задача сводится к построению минимального остовного дерева на полном графе, где вершинами являются точки, а весом ребра между двумя точками служит евклидово расстояние. Сначала строится список всех рёбер: двойной цикл перебирает все пары точек, вычисляет расстояние по теореме Пифагора и добавляет соответствующее ребро с весом в общий список. Полученный список сортируется по возрастанию веса.

Для построения остова применён алгоритм Краскала с системой непересекающихся множеств. Каждое множество изначально состоит из одной вершины. Рёбра перебираются в порядке возрастания веса; если концы очередного ребра принадлежат разным множествам, ребро включается в осто, его вес добавляется к общей сумме, а соответствующие множества объединяются. Процесс продолжается до тех пор, пока не будет добавлено $n-1$ ребро, после чего алгоритм завершается.

В системе непересекающихся множеств используется ранговая эвристика: при объединении двух множеств корнем становится то, у которого ранг больше. Это обеспечивает почти константное время операций поиска и объединения. Итоговый вес остова выводится с девятью знаками после запятой, что гарантирует требуемую точность. Сложность решения - $O(n^2 \log n)$ по времени за счёт сортировки рёбер и $O(n^2)$ по памяти для хранения полного графа.

Описание проведённых тестов

input.txt		output.txt	
1	4	1	3.000000000
2	0 0	2	
3	0 1		
4	1 0		
5	1 1		
6			

input.txt		output.txt	
1	5	1	7.064495102
2	0 0	2	
3	0 2		
4	1 1		
5	3 0		
6	3 2		
7			

```

13 > def test_minimal(): & MarieeMorte
14     check( task_num: 18, input_data: "1\n0 0\n", expected: "0.000000000")
15
16
17 > def test_minimal_two_points(): & MarieeMorte
18     check( task_num: 18, input_data: "2\n0 0\n3 4\n", expected: "5.000000000")
19
20
21 > def test_example_1(): & MarieeMorte
22     check( task_num: 18, input_data: "4\n0 0\n0 1\n1 0\n1 1\n", expected: "3.000000000")
23
24
25 > def test_example_2(): & MarieeMorte
26     check_mst( input_data: "5\n0 0\n0 2\n1 1\n3 0\n3 2\n", expected: "7.064495102", tol=1e-6)
27
28
29 > def test_maximal(): & MarieeMorte
30     n = 200
31     lines = [str(n)]
32     for i in range(n):
33         lines.append(f"{i} 0")
34     check_mst("\n".join(lines) + "\n", expected: "199.000000000", tol=1e-6)

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	time=0.085s	memory=188.0 KB
Первый пример из задачи	time=0.098s	memory=256.0 KB
Второй пример из задачи	time=0.094s	memory=248.0 KB
Верхняя граница диапазона значений входных данных из текста задачи	time=0.108s	memory=1.4 MB

Вывод по задаче

Был реализован алгоритм Краскала для нахождения минимальной суммарной длины отрезков, соединяющих заданные точки на плоскости. Использование системы непересекающихся множеств с ранговой эвристикой обеспечило эффективную проверку на циклы, а построение полного графа с последующей сортировкой позволило корректно обработать задачу при числе точек до двухсот.

Вывод по проделанной работе

В ходе выполнения лабораторной работы были изучены и реализованы основные алгоритмы работы с графами: обходы в глубину и в ширину, топологическая сортировка, поиск компонент сильной связности, алгоритмы Дейкстры, Беллмана-Форда и Краскала. Все задачи решены на Python с использованием итеративных реализаций обходов, что позволило корректно обрабатывать графы с числом вершин до ста тысяч без переполнения стека вызовов. Отдельное внимание было уделено работе с отрицательными весами: для обнаружения отрицательных циклов и классификации расстояний в задачах о валютах применён алгоритм Беллмана-Форда с инициализацией всех расстояний нулями. Все решения укладываются в заданные ограничения по времени и памяти.